SYSTEM DESIGN

1. Arquitetura de Segurança

1.1. Objetivo

A arquitetura de segurança da OrbiVet tem como objetivo proteger os dados pessoais, as contas, as informações dos animais e os registros de clínicas veterinárias, pet shops e demais prestadores de serviços, impedindo acessos indevidos e garantindo que cada usuário visualize e altere somente as informações permitidas para sua função.

1.2. Princípios de Segurança

A arquitetura será orientada pelos seguintes princípios:

- Confidencialidade: impedir que pessoas sem autorização consultem dados pessoais, informações de saúde dos animais ou registros privados.
- Integridade: impedir alterações, exclusões ou modificações indevidas nos dados dos usuários, animais, agendamentos e históricos.
- Disponibilidade: manter os dados e as funcionalidades acessíveis quando necessários, utilizando backups e mecanismos de recuperação.
- Privilégio mínimo: conceder a cada usuário apenas as permissões necessárias para realizar suas atividades.
- Privacidade desde a concepção: coletar somente os dados necessários, explicar suas finalidades e limitar seu compartilhamento.
- Rastreabilidade: registrar operações administrativas sensíveis para identificar quem realizou cada ação e quando ela ocorreu.

1.3. Autenticação e Gerenciamento de Contas

O sistema deverá validar os dados de cadastro, impedir contas duplicadas pelo mesmo e-mail e proteger o login contra tentativas excessivas. As senhas serão armazenadas por meio de algoritmos seguros de hash, nunca em texto puro.

A recuperação de senha utilizará mecanismos temporários e de uso único. As sessões terão prazo de validade e poderão ser revogadas em situações de risco, como troca de senha ou comprometimento da conta. Por utilizar uma API REST stateless, o backend não dependerá de sessões mantidas apenas na memória de uma instância.

1.4. Autorização e Controle de Acesso por Perfil

O acesso será definido conforme o perfil do usuário e sua relação com o recurso solicitado.

Perfil	Permissões
Tutor	Gerenciar seus animais, históricos, medicamentos, alimentação, lembretes e alertas.
Veterinário / Profissional	Acessar informações necessárias ao cuidado dos animais autorizados e consultar informações compartilhadas para atendimentos autorizados.
Clínica / Pet Shop	Gerenciar serviços, horários, agendamentos e acessar dados necessários ao atendimento.
ONG / Organização	Gerenciar campanhas e ações de apoio animal autorizadas.
Administrador	Realizar atividades de administração, suporte e moderação com privilégios controlados.

1.5. Proteção das APIs e dos dados

A comunicação utilizará HTTPS, e as APIs deverão validar entradas, restringir requisições abusivas e impedir acessos indevidos por manipulação de identificadores. As mensagens de erro não deverão expor informações internas.

Os dados deverão ser coletados conforme a necessidade da plataforma, com transparência sobre suas finalidades e compartilhamentos. O sistema deverá permitir consulta, correção e solicitação de exclusão quando aplicável, respeitando as obrigações legais de retenção.

Nos alertas de animais perdidos, o tutor deverá visualizar e confirmar quais informações serão públicas. A localização residencial exata não será divulgada automaticamente.

2. Modelagem do Banco de Dados e Integridade

2.1. Objetivo

O banco de dados do OrbiVet deverá centralizar as informações de usuários, animais, saúde, medicamentos, alimentação, lembretes, estabelecimentos, serviços, agendamentos e alertas de animais desaparecidos. A modelagem deverá garantir integridade, segurança, rastreabilidade e preservação dos históricos, evitando dados duplicados, vínculos inválidos e acessos não autorizados.

Entidades Banco de Dados:
veterinario; animal; prestador_servicos; dono; clinica; pet_shop; alerta_animais; vacinas; medicacoes; servicos; orcamento; exames; agendamento; usuario; prescricao e vacinacao.

Entidades Fortes:
usuario; dono; veterinario; prestador; clinica; pet_shop; servico; vacina; medicacao.

Entidades Fracas:
animal; exame; alerta_animais

Entidades Associativas:
vacinacao; prescricao; agendamento; orcamento; prestador_servicos.

2.2. Restrições de integridade

O banco deverá aplicar regras para evitar inconsistências:

- Unicidade: o e-mail de login não pode pertencer a duas contas diferentes.
- Obrigatoriedade: campos essenciais não podem ficar vazios.
- Integridade referencial: registros devem apontar para usuários, animais ou serviços existentes.
- Validação de valores: pesos, quantidades e preços não devem ser negativos quando aplicável.
- Estados válidos: medicamentos, lembretes, agendamentos e alertas devem respeitar estados definidos.
- Controle de acesso: o sistema deverá verificar o usuário, suas permissões e seu vínculo com cada registro.
- Transações: operações que alterem vários registros relacionados devem evitar atualizações parciais em caso de falha.

2.3. Exclusão e retenção de dados

As exclusões deverão preservar históricos relevantes, evitar registros órfãos e respeitar as obrigações de retenção aplicáveis.

- Contas - Permitir solicitação de exclusão e tratar os dados conforme as regras aplicáveis.
- Animais - Permitir arquivamento ou remoção da visualização ativa, preservando históricos necessários.
- Saúde e medicamentos - Evitar exclusão automática de históricos relevantes; permitir concluir ou interromper controles sem apagar registros anteriores.
- Agendamentos - Preservar o histórico de solicitações, cancelamentos e serviços concluídos quando necessário.
- Animais perdidos - Encerrar alertas quando o animal for encontrado, mantendo o histórico básico sem exibi-lo como ativo.
- Avistamentos e auditoria - Manter os registros pelo período necessário, com acesso restrito quando aplicável.

3. Escolha e Justificativa do Banco de Dados

Durante a discussão do grupo, foram consideradas duas opções para hospedar o banco de dados: PostgreSQL e MySQL, cada um com suas particularidades. O MySQL é mais simples de administrar e costuma ser indicado para aplicações web de uso geral, como sites com cadastro, login e muitas leituras. Seu desempenho tende a ser inferior em consultas complexas e em cenários que exigem recursos avançados.

Já o PostgreSQL, apesar de exigir mais configuração e conhecimento técnico, oferece maior integridade de dados, suporte a geolocalização por meio do PostGIS (“A extensão PostGIS adiciona ao PostgreSQL suporte a dados geoespaciais, permitindo consultas por proximidade e área com índices espaciais, essencial para o Mapa Pet), dados semiestruturados com JSONB e melhor desempenho em consultas complexas. Por esses motivos, e pelas necessidades do OrbiVet, como o Mapa Pet e o controle de agendamentos, o grupo optou pelo PostgreSQL.

Em relação à escalabilidade, o PostgreSQL permite atender aplicações de diferentes portes, desde sistemas pequenos até ambientes com maior volume de dados e operações simultâneas. Para o OrbiVet, isso representa a possibilidade de ampliar gradualmente a capacidade do banco de dados conforme novas clínicas, usuários e funcionalidades forem incorporados. Estratégias como criação de índices adequados, otimização de consultas, utilização de pooling de conexões por ferramentas como o PgBouncer e monitoramento do consumo de recursos podem contribuir para manter o desempenho. Caso a demanda cresça significativamente, também poderão ser avaliadas alternativas como replicação e separação de cargas de leitura. Um fator relevante na decisão foi o modelo de licenciamento. O MySQL é distribuído em duas edições: a Community, gratuita, e a Enterprise, paga e mantida pela Oracle. Diversos recursos de segurança e operação estão disponíveis apenas na edição Enterprise, como auditoria (Enterprise Audit), Já o PostgreSQL é distribuído sob uma licença livre, sem edições pagas, e oferece recursos equivalentes por meio de extensões gratuitas, como o pgaudit para auditoria, além de autenticação externa nativa e row-level security.

Diante desses fatores, o grupo optou pelo PostgreSQL por oferecer um conjunto de funcionalidades alinhado às necessidades da aplicação. Os mecanismos de integridade e controle de transações, os recursos de segurança, a flexibilidade de consulta e as possibilidades de expansão tornam essa tecnologia adequada para atender às funcionalidades previstas. Somados à licença permissiva e à ampla documentação disponível, esses aspectos justificam a escolha como uma solução capaz de sustentar o desenvolvimento e a evolução do sistema a longo prazo.

4. Infraestrutura

Infraestrutura PostgreSQL configurada em Multi-AZ para garantir a resiliência do sistema.Hardware: 4 vCPUs, 32 GB de memória e SSD (preferencialmente NVME) com IOPS garantidos. Gerenciamento de acessos PgBouncer configurado em modo transação para evitar o consumo de conexões do banco de dados na proporção que cresce a demanda pelo software. Arquitetura focada em performance.

Operações idempotentes por meio de INSERT ... ON CONFLICT para tratar erros de rede e retransmissões. Gerenciamento de sobrecarga (fluxos assíncronos) Filas (Redis Streams, SQS ou RabbitMQ) entre o banco e uma API para dados menos críticos, como notificações, logs e históricos. Trabalhadores consomem essas filas e fazem gravação em bloco. Dados necessários, como orçamentos e agendamentos, devem ser processados sincronizadamente.

Particionamento de dados Uso de PARTITION BY RANGE (via pg_partman) baseado em dados para tabelas de grande volume, como logs, histórico de Agendamento e Alerta_animais. Remoção ou arquivo de partições antigas para melhorar o desempenho do VACUUM.

Otimização do motor PostgreSQL Definição de shared_buffers em cerca de 25% da RAM, aumento do max_wal_size e configuração de checkpoint_timeout entre 10 e 15 minutos. Ativação de wal_compression. Configuração de Autovacuum intensivo para tabelas de alta atividade. Desativação do synchronous_commit apenas para registros de log, onde uma perda de milissegundos é aceitável.

Separação de leitura e escrita Uso de réplicas para o Mapa Pet, relatórios e consultas. Cache de buscas recorrentes no Redis. PostGIS com índices GiST para as funções geográficas do Mapa Pet.

Durabilidade e desastre Backups automáticos com PITR (via arquivo de WAL). Métricas de recuperação: RTO máximo de 30 minutos e RPO de até 5 minutos.

Testes e monitoramento Testes de carga com pgbench, JMeter ou k6 para verificar os limites definidos. Monitoramento via pg_stat_statements com monitoramento de travamentos, atrasos de replicação, IOPS e CPU. Infraestrutura de acesso.

O uso de chaves específicas e comandos INSERT ... ON CONFLICT garantem que requisições repetidas não criem duplicidade de dados. Controle de vazão (limitação de taxa).

A API usa um gateway para limitar o número de chamadas que um usuário pode fazer, protegendo o banco de dados de sobrecargas pontuais. Expansão lateral.

A API é stateless e usa um balanceador de carga, permitindo que o número de servidores cresça com escalonamento automático conforme necessário. Um PostgreSQL gerenciado com Multi-AZ, PgBouncer, gravação em lote, fila para dados não críticos, particionamento por dados e réplicas de leitura suporta 1 milhão de escritas sem sharding.

5. Projetar falhas, recuperação e disponibilidade

5.1. Objetivo

Definir como a OrbiVet deverá se comportar quando ocorrerem falhas no banco de dados, no armazenamento de arquivos, nos mapas, nas notificações ou nas APIs externas. O objetivo é reduzir a indisponibilidade, evitar a perda ou duplicação de informações e garantir que o sistema consiga recuperar suas operações com segurança.

A estratégia deve priorizar a preservação dos dados dos tutores e dos animais, mantendo o histórico de saúde, os medicamentos, os agendamentos e os alertas de animais perdidos consistentes, mesmo quando algum serviço estiver temporariamente indisponível.

5.2. Tratamento de falhas por serviço

- Serviço - Comportamento esperado em caso de falha
- Banco de dados - Informar que a operação está temporariamente indisponível, registrar o erro e tentar restabelecer a conexão sem confirmar operações que não foram concluídas.
- Armazenamento de arquivos - Informar quando fotos ou documentos não puderem ser enviados, permitir uma nova tentativa e evitar registrar o envio como concluído antes da confirmação.
- Mapas e localização - Exibir uma mensagem de indisponibilidade e manter disponíveis as demais funções da plataforma.
- Notificações - Registrar notificações pendentes e tentar enviá-las novamente, evitando duplicações.
- APIs externas - Aplicar limite de tempo de espera (timeout), tratar erros e impedir que uma falha externa comprometa os dados já registrados internamente.

A indisponibilidade de um recurso secundário, como mapas ou notificações, não deve interromper funcionalidades que não dependam diretamente dele.

5.3. Tentativas de reenvio

O sistema deverá realizar novas tentativas quando uma operação falhar por motivos temporários, como instabilidade de rede ou indisponibilidade de um serviço externo.

- Reenvio automático: utilizar novas tentativas com intervalos crescentes entre elas, evitando sobrecarregar o serviço que apresentou falha.

- Limite de tentativas: estabelecer uma quantidade máxima de reenvios. Caso o limite seja atingido, registrar a falha e permitir nova tentativa posterior, quando aplicável.
- Operações duplicadas: garantir que repetir uma solicitação não provoque efeitos duplicados. Por exemplo, uma mesma solicitação de agendamento não deve criar duas reservas, e uma notificação reenviada não deve ser registrada como dois eventos distintos.
- Timeouts: definir um tempo máximo de espera para respostas do banco, do armazenamento e das APIs externas, evitando operações bloqueadas indefinidamente.

5.4. Recuperação de desastres e disponibilidade

Em situações mais graves, como corrupção do banco, falha de infraestrutura ou indisponibilidade prolongada, a equipe deverá seguir um procedimento documentado de recuperação:

1. Identificar e avaliar a falha.
2. Isolar o problema, quando necessário, para impedir danos adicionais.
3. Restaurar o banco ou os arquivos a partir de cópias válidas.
4. Verificar a integridade dos dados e os relacionamentos entre os registros.
5. Restabelecer os serviços prioritários e verificar seu funcionamento.
6. Identificar possíveis operações pendentes, como notificações ou atualizações que precisam ser processadas novamente.
7. Registrar a ocorrência, suas causas e as medidas adotadas.

6. Definir a Estratégia de Testes

6.1. Objetivo

A estratégia de testes da OrbiVet tem como objetivo garantir que as funcionalidades sejam executadas corretamente e que os dados permaneçam íntegros.

Os testes serão aplicados durante o desenvolvimento e antes da entrega de cada versão, contemplando os fluxos críticos do sistema, a comunicação entre os componentes, a segurança, o tratamento de falhas e o desempenho. A estratégia considera os requisitos do projeto relacionados à autorização de acesso, à integridade dos registros e à continuidade das operações.

6.2. Cenários prioritários

Os testes deverão contemplar os principais fluxos de negócio da plataforma:

- Cadastro e autorização de animais: validar dados obrigatórios, impedir acessos não autorizados e confirmar que permissões revogadas deixem de valer.
- Lembretes de saúde: verificar criação, alteração, datas inválidas e prevenção de notificações duplicadas.
- Alertas de animais perdidos: testar publicação, encerramento e preservação do histórico.
- Busca por clínicas e serviços: validar filtros, resultados vazios e funcionamento quando os mapas estiverem indisponíveis.
- Solicitação de orçamento: verificar criação da solicitação e garantir que somente o prestador autorizado possa responder.
- Agendamentos: testar disponibilidade de horários, conflitos entre solicitações simultâneas, cancelamentos e atualização de status.
- Também deverão ser testados o tratamento de erros, a consistência das operações no banco de dados e a restauração dos backups.

6.3. Automação dos testes

Momento	Testes previstos
A cada alteração do código	Testes unitários e validações das regras de negócio.
Durante a integração do código	Testes de integração, contratos das APIs e integridade dos dados.
Antes da publicação	Testes de desempenho, falhas simuladas e verificação dos procedimentos de recuperação.

7. Planejar Monitoramento e Operação

7.1. Objetivo

O monitoramento e a operação da OrbiVet têm como objetivo acompanhar o funcionamento da plataforma, identificar falhas rapidamente e fornecer informações para que a equipe técnica consiga investigar e resolver problemas em produção.

A estratégia deverá contemplar o acompanhamento da API, do banco de dados, das notificações, do armazenamento de arquivos e das integrações externas. Também deverá garantir que os erros relevantes sejam registrados e que a equipe seja alertada quando ocorrerem problemas que possam comprometer a disponibilidade, a segurança ou a integridade dos dados.

7.2. Logs estruturados

A OrbiVet deverá utilizar logs estruturados para registrar eventos importantes em um formato padronizado, facilitando a busca, a filtragem e a investigação de ocorrências.

Cada registro deverá conter, quando aplicável:

- Data e horário: momento em que o evento ocorreu, com referência temporal padronizada.
- Nível do evento: informação, aviso, erro ou falha crítica.
- Componente: API, banco de dados, notificações, armazenamento ou integração externa.
- Identificador da requisição: para acompanhar uma operação entre diferentes componentes.
- Operação executada: por exemplo, cadastro de animal, criação de lembrete ou atualização de agendamento.
- Resultado: sucesso, falha ou operação parcialmente concluída.
- Código de erro: identificação padronizada para facilitar a análise.
- Duração: tempo necessário para executar a operação, quando relevante.

Os logs deverão evitar senhas, tokens de autenticação, dados pessoais desnecessários e informações clínicas que não sejam indispensáveis à investigação. O acesso aos registros deverá ser restrito e controlado.

7.3. Métricas e acompanhamento dos serviços

A equipe deverá acompanhar métricas operacionais para identificar lentidão, indisponibilidade e falhas antes que afetem muitos usuários. O que cada componente deve monitorar:

- API - Disponibilidade, volume de requisições, tempo de resposta, taxa de erros e respostas de timeout.
- Banco de dados - Tempo das consultas, conexões utilizadas, consultas lentas, falhas de conexão e uso de recursos.
- Notificações - Quantidade de mensagens pendentes, taxa de envio bem-sucedido, falhas, tentativas de reenvio e atrasos.
- Armazenamento - Falhas de envio e leitura de arquivos, espaço disponível e tempo de resposta.
- Integrações externas - Disponibilidade, latência, erros e timeouts de serviços como mapas.
- Infraestrutura - Uso de CPU e memória, espaço em disco e disponibilidade dos recursos necessários à aplicação.
- Segurança - Tentativas repetidas de autenticação, acessos negados e atividades administrativas relevantes.

7.4. Alertas operacionais

Os alertas deverão notificar a equipe quando houver sinais de falha ou degradação dos serviços. Sempre que possível, deverão incluir o componente afetado, o horário, a gravidade e informações suficientes para iniciar a investigação.

Os limites que disparam os alertas deverão ser definidos após a obtenção de métricas reais e a identificação das metas de disponibilidade e desempenho. Isso evita tanto alertas excessivos quanto a demora na identificação de problemas.

7.5. Procedimento para investigação de problemas em produção

Quando uma falha for identificada, a equipe deverá seguir um procedimento padronizado para reduzir o tempo de diagnóstico e evitar que uma correção provoque novos problemas.

1. Identificar o incidente: verificar o alerta recebido, o componente afetado, o horário de início e a gravidade.
2. Avaliar o impacto: identificar quais funcionalidades e usuários foram afetados e se existe risco para os dados.
3. Investigar a causa: consultar logs, métricas, rastreamentos e alterações recentes no sistema.

4. Conter o problema: aplicar medidas seguras, como desativar temporariamente uma integração instável ou utilizar uma alternativa disponível.
5. Corrigir e validar: implementar a correção e verificar se a funcionalidade voltou a operar corretamente.
6. Recuperar operações pendentes: reprocessar tarefas e notificações quando necessário, evitando duplicidades.
6.7. Registrar o incidente: documentar causa, impacto, duração, solução aplicada e ações preventivas.

Se houver suspeita de perda ou corrupção de dados, a equipe deverá priorizar a preservação das informações e seguir os procedimentos de recuperação definidos na etapa 4 do System Design.

8. Justificativa do Backend Stateless para Escalabilidade

8.1. Arquitetura Stateless

O OrbiVet utilizará um backend REST stateless para facilitar a escalabilidade do sistema e permitir que diferentes servidores processem as requisições dos usuários de forma independente.

Em uma arquitetura stateless, o servidor não armazena informações de sessão do usuário em sua própria memória para atender às próximas requisições. Cada solicitação contém os dados necessários para ser processada, enquanto as informações que precisam ser mantidas, como cadastros de animais, usuários e históricos de saúde, ficam armazenadas no banco de dados.

Na prática, isso significa que um tutor conseguirá consultar os dados do seu pet em uma instância do backend e, na próxima requisição, ser atendido por outra, sem que isso prejudique o funcionamento da aplicação.

A autenticação seguirá esse modelo, utilizando tokens JWT enviados nas requisições e validados pelo backend, sem depender de sessões armazenadas localmente.

8.2. Como o Stateless Ajudará na Escalabilidade

A principal vantagem dessa arquitetura é a escalabilidade horizontal, ou seja, a possibilidade de adicionar novas instâncias da aplicação conforme a quantidade de acessos aumenta.

Para isso, o OrbiVet vai usar um Load Balancer, que vai ser responsável por distribuir as requisições entre os servidores disponíveis.

Por exemplo, quando muitos usuários acessarem o OrbiVet ao mesmo tempo para consultar animais perdidos, visualizar históricos de vacinação ou realizar agendamentos, essas requisições não ficarão armazenadas em um único servidor.

Como as instâncias serão independentes, a carga será distribuída entre elas. Com base no aumento da demanda, novas instâncias serão adicionadas para aumentar a capacidade de processamento.

Outro benefício será a disponibilidade. Se uma instância apresentar problemas, o balanceador encaminhará novas requisições para outras instâncias ativas, evitando que a falha de um único servidor interrompa todo o atendimento.

8.3. Relação com a Proporção de Leitura e Escrita

No OrbiVet, as leituras acontecem quando o usuário consulta o perfil de um animal, vê as vacinas registradas, procura clínicas ou busca um pet perdido. Já as escritas acontecem quando alguém cadastra um animal, atualiza seus dados, registra uma vacina ou marca uma consulta.

A previsão é que, a cada 3 operações realizadas no sistema, aproximadamente 2 sejam leituras e 1 seja uma escrita. Por isso, o OrbiVet precisa conseguir atender várias requisições ao mesmo tempo sem comprometer seu funcionamento.

Para ajudar nisso, o backend stateless permitirá distribuir as requisições entre diferentes instâncias da API. Assim, em vez de depender de um único servidor para processar tudo, o sistema poderá dividir esse trabalho e adicionar mais instâncias conforme a quantidade de acessos aumentar.

Na prática, quando alguém consultar as informações de um animal, a API vai buscar os dados no banco e retornar o resultado. Quando houver um cadastro ou alguma atualização, ela vai validar as informações e enviá-las ao banco para serem salvas.

Mas só aumentar o número de instâncias não resolve tudo, já que o banco de dados também pode ficar sobrecarregado com tantas operações. Por isso, o projeto também vai utilizar cache, réplicas de leitura e melhorias nas consultas ao banco.

O cache será usado para evitar que o sistema precise buscar as mesmas informações no banco toda vez que alguém fizer uma consulta. Já as réplicas de leitura ajudarão a dividir o volume de consultas, deixando o banco principal com menos trabalho. Os cadastros e as atualizações continuarão sendo feitos no banco principal, para manter os dados corretos.

Com isso, a ideia é que o OrbiVet consiga atender mais usuários sem depender apenas de um aumento na capacidade dos servidores. Cada solução vai ajudar de uma forma diferente, tanto na distribuição das requisições quanto na redução da sobrecarga do banco de dados.

8.4. Limitações e Cuidados

Mesmo com várias instâncias do backend funcionando, todas ainda vão depender dos mesmos serviços, principalmente do banco de dados. Por isso, é importante garantir que ele consiga lidar com a quantidade de consultas e gravações feitas pelo sistema.

Também precisamos pensar nos momentos em que muitos usuários acessam o sistema ao mesmo tempo, como quando um animal perdido é divulgado e várias pessoas entram na plataforma para buscar informações.

Por esse motivo, não tem que apenas distribuir as requisições entre as instâncias. Também vai ser necessário acompanhar o desempenho do banco de dados, o número de conexões, o tempo de resposta e o consumo de recursos da aplicação.

Outro ponto importante é evitar que informações necessárias para o funcionamento do sistema fiquem armazenadas apenas na memória de uma instância. Porque, se ela for reiniciada ou substituída, esses dados podem ser perdidos.

Para garantir que tudo funcione, vai ser realizado vários testes de carga, simulando diferentes quantidades de usuários acessando o sistema ao mesmo tempo. Com esses testes, será possível identificar futuros problemas de desempenho e descobrir se adicionar mais instâncias realmente vai ajudar a melhorar o funcionamento do OrbiVet.

8.5. Conclusão

A escolha de um backend REST stateless para o OrbiVet foi pensada para facilitar o crescimento do sistema e permitir que as requisições sejam distribuídas entre diferentes instâncias da aplicação.

Com isso, o sistema poderá lidar melhor com uma quantidade de 1 milhão de escritas para cada 2 milhões de leituras, permitindo aumentar a capacidade do backend conforme a demanda crescer. Como ele não depende de sessões armazenadas em um servidor específico, fica mais fácil distribuir as requisições e adicionar novas instâncias quando necessário.

Assim, o uso de um backend stateless ajuda a preparar o OrbiVet para crescer, distribuindo melhor as requisições e reduzindo a dependência de servidores específicos. Dessa forma, o sistema terá uma estrutura mais preparada para lidar com o aumento de usuários e operações ao longo do tempo.

9. Arquitetura de Comunicação em Tempo Real via WebSocket

O OrbiVet utilizará uma arquitetura de comunicação em tempo real para permitir o envio de notificações relacionadas a eventos relevantes da plataforma, com destaque para a funcionalidade de animais desaparecidos.

O fluxo terá início quando um usuário realizar uma operação por meio do frontend, como registrar um possível avistamento. A requisição será encaminhada à API REST, responsável por validar os dados, verificar as permissões e aplicar as regras de negócio. Após a confirmação da persistência do registro no banco de dados, o backend poderá publicar um evento interno, como sighting.created.

O evento será encaminhado ao serviço de WebSocket, que identificará as conexões ativas dos destinatários autorizados e enviará uma mensagem em tempo real ao frontend correspondente. A interface poderá apresentar uma notificação ao tutor responsável pelo alerta, sem exigir que ele atualize manualmente a página.

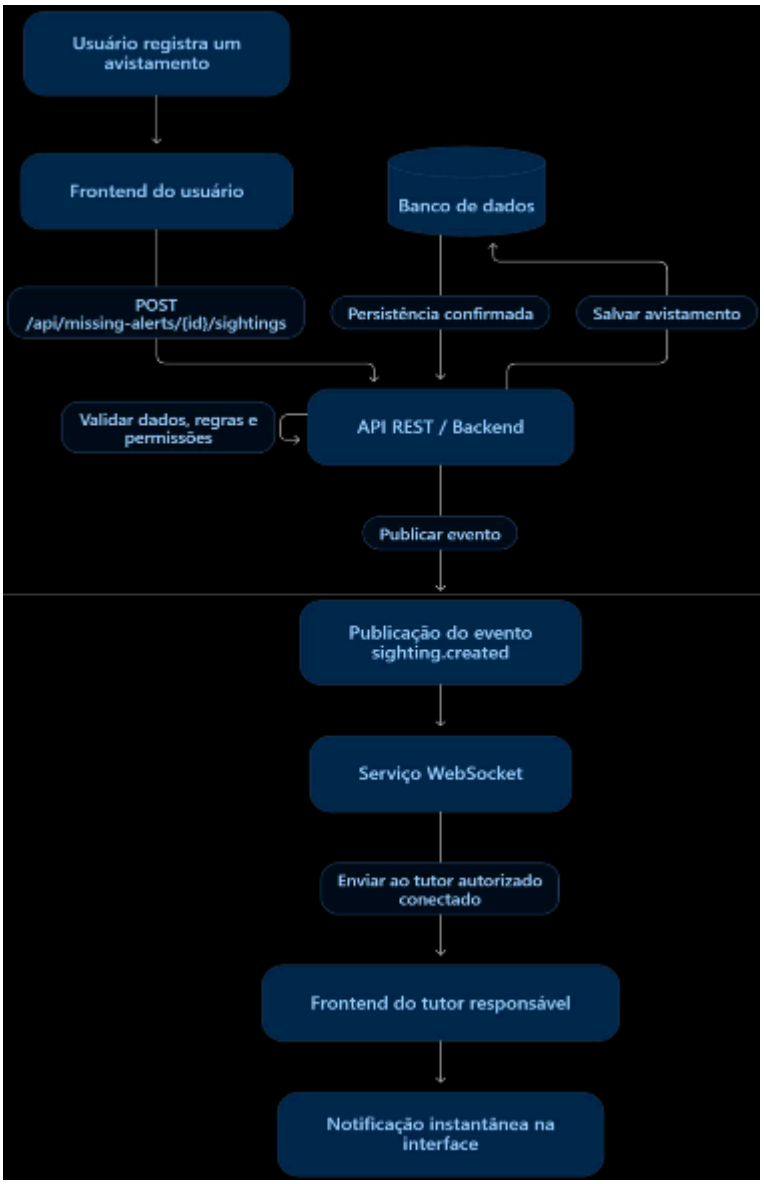
A conexão WebSocket será estabelecida por meio de um handshake HTTP e permitirá a troca bidirecional de mensagens enquanto permanecer ativa. O serviço deverá controlar a autenticação da conexão, a autorização dos destinatários e o encerramento ou a reconexão de clientes.

A utilização de WebSockets complementará a API REST, sem substituí-la. As operações de criação, consulta e atualização dos recursos continuarão sendo realizadas pela API. O WebSocket será utilizado para distribuir avisos de eventos ocorridos na aplicação.

Caso o destinatário esteja desconectado, o sistema deverá preservar a possibilidade de consultar as notificações posteriormente. Para isso, os registros de notificação deverão ser persistidos no banco de dados, e a interface poderá recuperá-los por meio da API REST após reconectar-se. O envio em tempo real será um mecanismo de comunicação imediata, não a única forma de acesso às informações.

Essa arquitetura busca reduzir a necessidade de consultas periódicas e melhorar a rapidez com que os responsáveis recebem informações relevantes sobre possíveis avistamentos e atualizações dos alertas de desaparecimento.

Na teoria como isso funcionaria?



Explicação:

- Um usuário registra um possível avistamento no frontend.
- O frontend envia uma requisição HTTP para a API REST.
- O backend valida os dados, verifica as permissões e aplica as regras de negócio.
- O registro do avistamento é salvo no banco de dados.
- Após a confirmação da persistência, o backend publica um evento interno.
- O serviço WebSocket identifica os destinatários conectados e autorizados.
- O tutor responsável recebe a notificação na interface.

E como isso funcionaria em nosso sistema?

1. Um usuário registra um possível avistamento de um animal perdido por meio do frontend do OrbiVet.

2. O frontend envia uma requisição HTTP POST para o endpoint /api/missing-alerts/{id}/sightings, encaminhando os dados do avistamento à API REST.

3. O backend recebe a requisição, valida os dados, verifica as permissões do usuário e aplica as regras de negócio necessárias.

4. Após a validação, o avistamento é salvo no banco de dados. O sistema também registra as notificações necessárias para permitir que sejam consultadas posteriormente, mesmo que o destinatário esteja desconectado.

5. O banco de dados confirma a persistência do avistamento. Essa confirmação permite que o backend prossiga com a publicação do evento.

6. O backend publica o evento interno sighting.created, indicando que um novo avistamento foi registrado com sucesso.

7. O evento é encaminhado ao serviço WebSocket, responsável pela comunicação em tempo real com os clientes conectados.

8. O serviço WebSocket verifica se o tutor responsável pelo alerta está conectado e autorizado a receber a notificação, respeitando as permissões de acesso e as regras de privacidade.

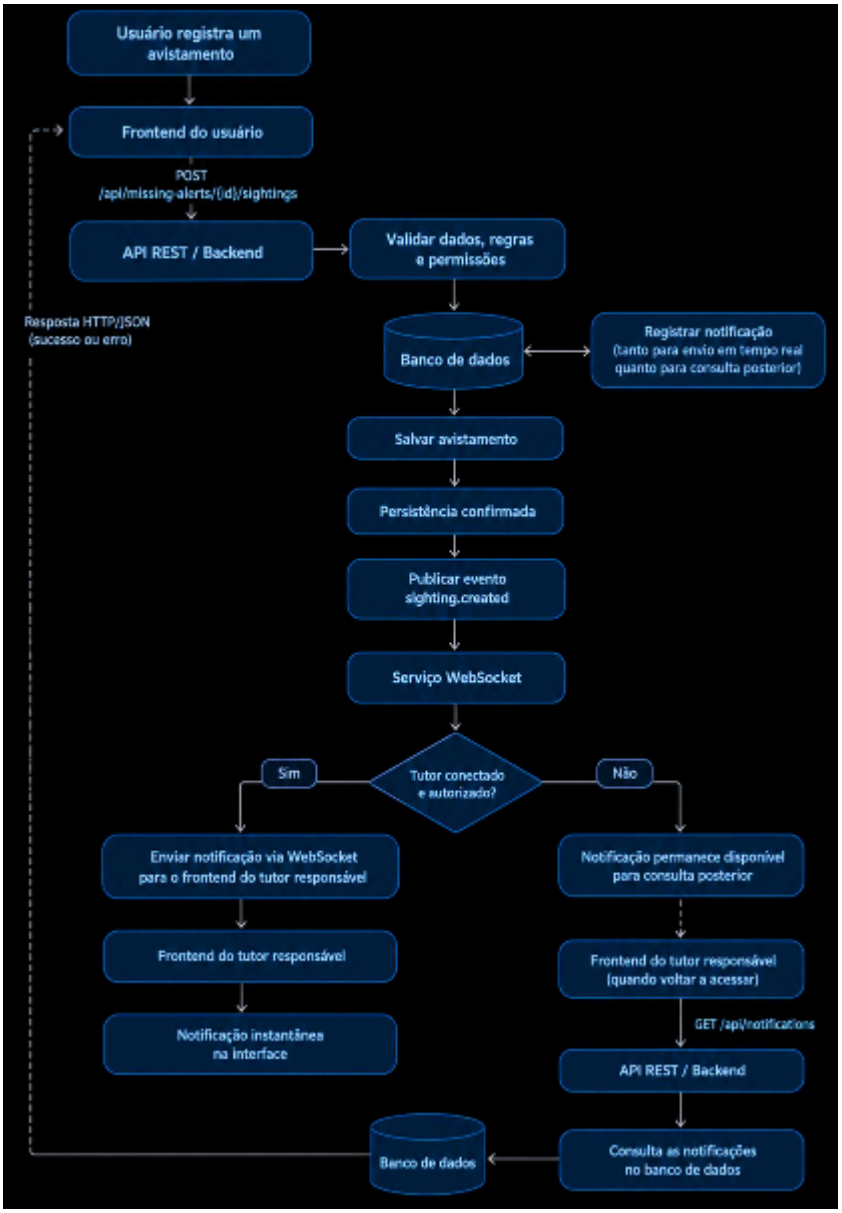
9. Se o tutor estiver conectado e autorizado, o serviço WebSocket envia a notificação ao frontend do tutor responsável, permitindo que a informação seja exibida instantaneamente na interface.

10. Se o tutor não estiver conectado, a notificação permanece disponível para consulta posterior, conforme o mecanismo de persistência representado no diagrama.

11. Quando o tutor voltar a acessar o sistema, o frontend poderá enviar uma requisição HTTP GET para o endpoint /api/notifications, solicitando as notificações disponíveis.

12. A API REST consulta as notificações armazenadas no banco de dados e devolve os resultados ao frontend, que poderá exibi-los ao tutor.

13. Caso ocorra um erro durante o processamento da requisição inicial, a API REST deverá retornar uma resposta HTTP apropriada, indicando sucesso ou falha da operação ao frontend



10. Arquitetura do Backend REST Stateless

O backend do OrbiVet será organizado segundo o estilo arquitetural REST, utilizando o protocolo HTTP para a comunicação entre o frontend e os serviços da aplicação. Os recursos do sistema, como animais, alertas de desaparecimento, avistamentos e notificações, serão disponibilizados por meio de endpoints específicos, com operações definidas conforme a finalidade de cada recurso.

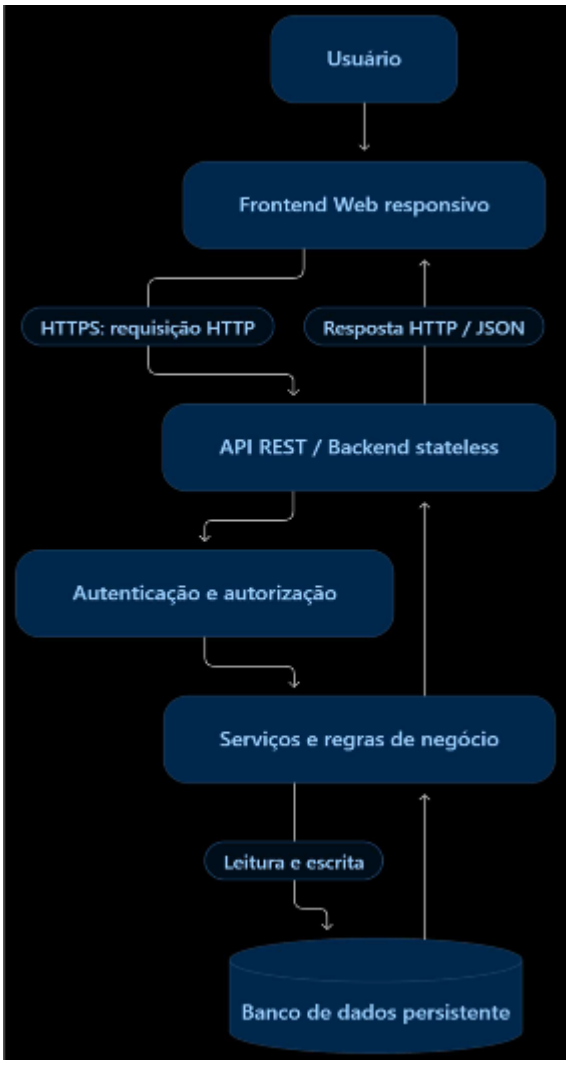
A arquitetura adotará o princípio stateless, ao qual cada requisição/solicitação deverá conter as informações necessárias para seu processamento, incluindo as credenciais de autenticação quando a operação exigir acesso protegido. O servidor não dependerá de uma sessão de usuário armazenada em sua memória entre requisições. A identidade e as permissões serão verificadas a cada operação protegida.

As regras de negócio serão executadas no backend, responsável por validar os dados recebidos, verificar as permissões do usuário e controlar as operações permitidas. Após o processamento, os dados serão consultados ou persistidos no banco de dados, e o resultado será devolvido ao cliente por meio de uma resposta HTTP.

A adoção dessa arquitetura favorece a escalabilidade e a manutenção do sistema, pois permite distribuir requisições entre múltiplas instâncias do backend sem depender de uma sessão mantida em uma instância específica, evitando também que em casos de problemas em instâncias as sessões do usuário não se percam. Os dados permanentes da aplicação continuarão armazenados no banco de dados, independentemente do princípio stateless.

A comunicação em produção deverá utilizar HTTPS, e os mecanismos de autenticação e autorização deverão proteger os recursos privados, respeitando os perfis dos usuários e as regras de negócio do OrbiVet.

Exemplo do sistema diagramado:



E como funciona esse sistema?

- Frontend Web responsivo: permite que tutores, clínicas e demais usuários interajam com a plataforma.
- API REST: recebe as requisições HTTP e retorna respostas estruturadas, normalmente em JSON.
- Autenticação e autorização: verifica a identidade do usuário e se ele possui permissão para realizar a operação.
- Serviços e regras de negócio: executam as operações do sistema, como cadastrar animais, publicar alertas e registrar avistamentos.
- Banco de dados: armazena os registros persistentes, mantendo os vínculos entre usuários, animais e alertas por exemplo.
- Resposta HTTP: informa o resultado da operação ao frontend, incluindo os dados solicitados ou o resultado da alteração.

11. Documentação de Consumo da API REST

11.1. Padrão de Comunicação e Segurança

- Autenticação Stateless (JWT): Como o servidor não guardará estado, o frontend (web e mobile) enviará as credenciais de login (POST/api/auth/login) e receberá um token. Todas as requisições subsequentes incluirão esse token no cabeçalho (Header Authorization) para validar a identidade.

- Criptografia: As senhas enviadas nos formulários de cadastro nunca trafegarão ou serão salvas em texto puro no banco de dados.

- Controle de Acesso Baseado em Perfis (RBAC): O backend validará o token para garantir regras de autorização rígidas, assegurando que clínicas e prestadores acessem apenas os dados do animal estritamente necessários para o serviço prestado.

- Endpoints de Adequação à LGPD: O aplicativo consumirá rotas específicas para que o usuário possa gerenciar, corrigir e excluir seus dados, bem como controlar permissões de localização.

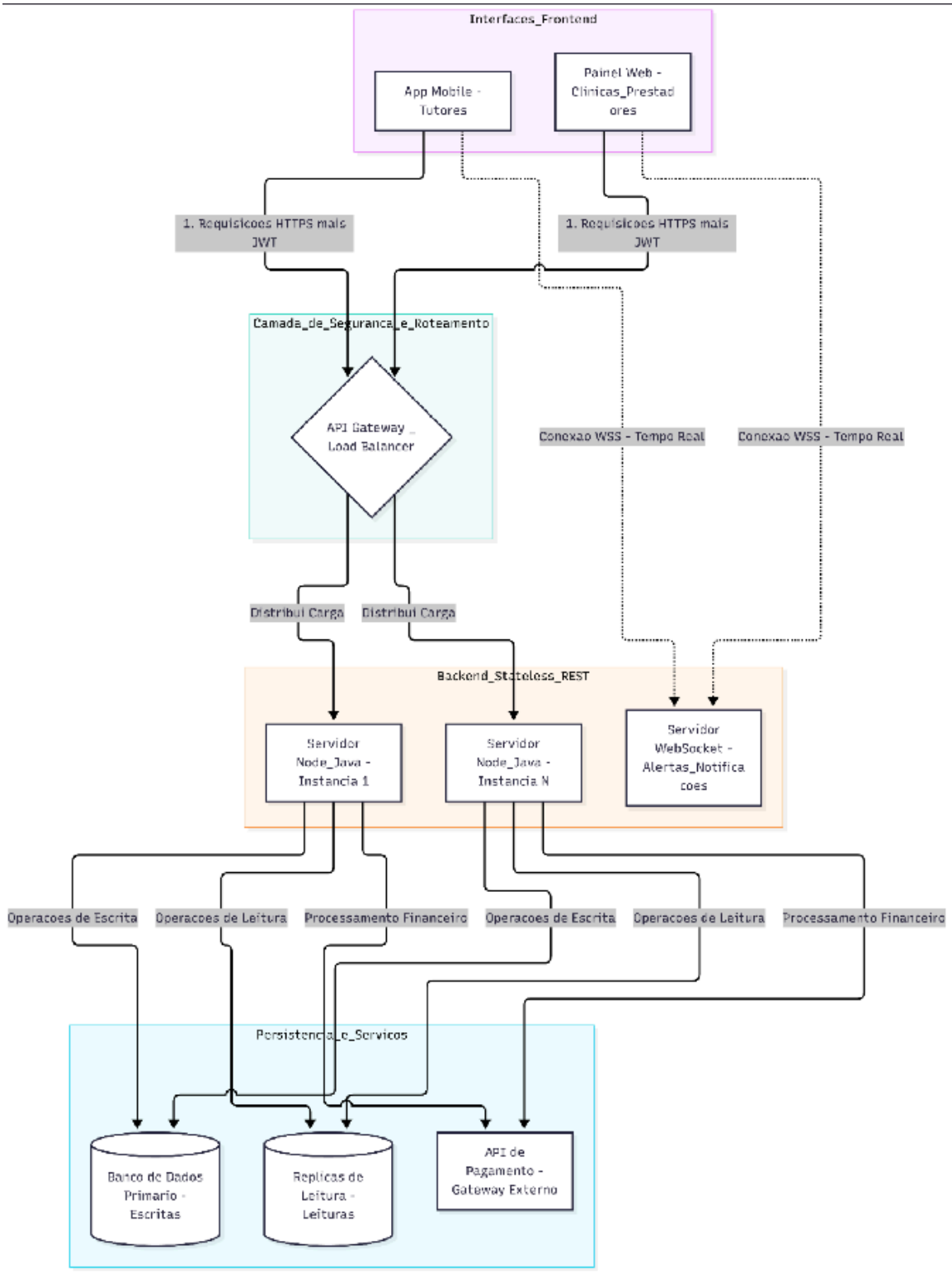
11.2. Detalhamento das Requisições - App Mobile (Tutores)

- Cadastro de Usuário e Pet:
  - POST/api/tutores: Envia o payload com os dados do tutor (nome, cpf, telefone, email, senha).
  - POST/api/animais: Envia os dados do pet (idDono, espécie, raça, idade, peso, características físicas, foto).
- Consumo do Histórico Centralizado:
  - GET/api/animais/{idAnimal}/historico: O app consome este endpoint para montar o painel unificado. O backend retornará um JSON agregando os arrays de vacinas, vermífugos, medicamentos e alergias vinculados ao pet.
- Alerta de Animais Perdidos:
  - POST/api/alertas: O app enviará um payload contendo o idAnimal, foto, características, data da ocorrência e as coordenadas geográficas (latitude e longitude) da última localização.
  - Essa requisição servirá de gatilho para o backend disparar notificações push em tempo real (via Websocket/FCM) para os usuários que estiverem no raio de distância do desaparecimento.
- Agendamentos e Mapa Pet:

    - GET/api/estabelecimentos/proximos: O app consome este endpoint enviando sua geolocalização atual para desenhar os "pins" de clínicas e hospitais na integração com a API de mapas (ex: Google Maps).
    - POST/api/agendamentos: Envia o idAnimal, idServico e idClinica_Prestador para reservar banho, tosa ou hospedagem.

11.3. Detalhamento das Requisições - Painel Web (Clínicas/Veterinários)

- Registro de Consultas e Exames:
    - POST /api/exames: A clínica enviará o payload vinculando o idAnimal e idVeterinario, além do tipo de exame, data de realização e o resultado.
- Prescrições e Tratamentos (Gatilho para o Despertador):
    - POST /api/prescricoes: O veterinário enviará via painel web os dados do tratamento, contendo o idMedicacao, dosagem, horários de administração (frequência) e duração do tratamento.
    - O envio desses dados criará os registros no banco que alimentarão a lógica de "despertador" no aplicativo mobile do tutor, notificando-o no horário exato da medicação.
- Saúde Preventiva (Vacinas e Vermífugos):
    - POST /api/vacinacoes: Registra a aplicação enviando o lote, dose e a dataProximaDose.
    - Assim como nos medicamentos, o envio da dataProximaDose (tanto para vacinas quanto para vermífugos) alimenta diretamente o módulo de lembretes preventivos do sistema.


Interfaces_Frontend
App Mobile - Tutores
Painel Web - Clinicas_Prestadores
1. Requisicoes HTTPS mais JWT
1. Requisicoes HTTPS mais JWT
Camada_de_Seguranca_e_Roteamento
API Gateway _ Load Balancer
Conexao WSS - Tempo Real
Conexao WSS - Tempo Real
Distribui Carga
Distribui Carga
Backend_Stateless_REST
Servidor Node_Java - Instancia 1
Servidor Node_Java - Instancia N
Servidor WebSocket - Alertas_Notificacoes
Operacoes de Escrita
Operacoes de Leitura
Processamento Financeiro
Operacoes de Escrita
Operacoes de Leitura
Processamento Financeiro
Persistencia_e_Servicos
Banco de Dados Primario - Escritas
Replicas de Leitura - Leituras
API de Pagamento - Gateway Externo

12. OrbiVet — Justificativa Técnica da Arquitetura e Infraestrutura

12.1.Introdução

A definição da stack do OrbiVet foi guiada pelas necessidades operacionais do sistema, custos de hospedagem, curva de aprendizado e facilidade de manutenção a longo prazo.
O escopo abrange múltiplos módulos — gestão de usuários, animais, clínicas, agendamentos, prontuários e pagamentos. Por isso, priorizamos tecnologias que ofereçam consistência de dados, segurança das informações e facilidade para futuras alterações.
A arquitetura adota Java com Spring Boot no back-end, React com TypeScript no front-end web, React Native no mobile e PostgreSQL como banco de dados principal. A infraestrutura será centralizada na AWS.

12.2 Tecnologias de desenvolvimento

Back-end — Java e Spring Boot

Optamos por Java devido à sua maturidade, estabilidade e forte presença no ecossistema de aplicações corporativas. A vasta documentação e o suporte da comunidade facilitam o desenvolvimento diário.

O Spring Boot agiliza a estruturação da API REST, que centraliza as regras de negócio e atende aos clientes web e mobile.

A segurança e a persistência contam com ferramentas do ecossistema: o Spring Security permite implementar autenticação e controle de acesso (tutores, veterinários e administradores), enquanto o Spring Data JPA facilita a integração com o banco relacional.

Em termos de desempenho, a JVM oferece recursos para lidar com concorrência e demandas simultâneas. Em contrapartida, o consumo de RAM pode ser elevado, exigindo dimensionamento adequado da instância para evitar custos desnecessários.

A organização interna em pacotes e camadas com responsabilidades separadas simplifica a manutenção dos fluxos de agendamentos, orçamentos e serviços veterinários.

Front-end Web — React e TypeScript

A interface web utiliza React com foco em componentes reutilizáveis, agilizando a criação de formulários padronizados, menus e tabelas de dados.
O TypeScript permite verificar os tipos de dados durante o desenvolvimento, ajudando a identificar erros antecipadamente e deixando mais clara a estrutura das informações recebidas da API.

Front-end Mobile — React Native e Expo

O aplicativo mobile foi pensado em React Native com Expo para compartilhar grande parte do código entre Android e iOS, reduzindo o trabalho necessário para manter duas aplicações nativas separadas.
Como o ecossistema compartilha conceitos com o React web, a equipe pode aproveitar conhecimentos já utilizados no desenvolvimento da interface web, mantendo a comunicação com a mesma API do back-end.

Banco de dados — PostgreSQL

O PostgreSQL foi escolhido por ser um banco relacional de código aberto, sem custos de licença e com recursos adequados para trabalhar com dados estruturados e relacionados.

No OrbiVet, as entidades são fortemente relacionadas: animais pertencem a tutores e acumulam histórico de vacinas, exames, prescrições e consultas. O uso de chaves primárias e estrangeiras ajuda a manter a integridade desses relacionamentos.

O suporte a transações ACID ajuda a manter a consistência dos dados, especialmente em operações financeiras, registros de pagamentos e agendamentos.

Conforme o volume de dados aumentar, o banco oferece recursos como índices e otimização de consultas, que podem ser utilizados para melhorar o desempenho.

Infraestrutura Cloud — AWS

A hospedagem inicial utilizará a Amazon Web Services (AWS), tendo o Amazon Lightsail como ponto de partida.

O Lightsail oferece servidores virtuais com custos mensais previsíveis, simplificando o orçamento inicial da infraestrutura sem a necessidade de gerenciar diversos serviços AWS separadamente.
A aplicação será executada em uma instância Linux, com um banco PostgreSQL gerenciado e um serviço de armazenamento de objetos para fotos dos animais e documentos.
A principal vantagem é permitir a disponibilização da primeira versão do sistema sem depender de infraestrutura física própria, além da possibilidade de aumentar os recursos do servidor conforme a quantidade de usuários crescer.
Como limitação, a utilização de uma única instância não garante alta disponibilidade. Para reduzir esse risco, serão necessárias rotinas de backup, monitoramento e configurações adequadas de segurança.

Implantação e organização do sistema

O uso do Docker padroniza o ambiente de execução, reduzindo diferenças entre as máquinas de desenvolvimento e o servidor de produção. O Docker Compose facilita a organização dos serviços utilizados no ambiente local.
Adotamos o padrão de monólito modular, no qual as funcionalidades são organizadas por módulos dentro de uma mesma aplicação back-end. Essa escolha evita a complexidade de gerenciar diversos microserviços, algo que não seria necessário nesta fase inicial do projeto.
Em relação à segurança, a comunicação utilizará HTTPS, autenticação por token e controle de permissões. Os webhooks do gateway de pagamentos deverão passar por validações de autenticidade, seguindo os mecanismos disponibilizados pelo serviço escolhido, para evitar alterações indevidas no status das transações.

12.3 Considerações finais

A stack foi definida considerando o equilíbrio entre desempenho, custos, segurança e facilidade de desenvolvimento e manutenção.
Java e Spring Boot ficam responsáveis pelas regras de negócio, React e React Native pelas interfaces, PostgreSQL pelo armazenamento dos dados e AWS pela hospedagem da aplicação.
Com Docker e a organização em módulos, a implantação e a manutenção ficam mais simples, permitindo ampliar a estrutura conforme o sistema crescer.

13. Especificação Técnica da Integração com API de Pagamentos

O objetivo é documentar como o OrbiVet poderá iniciar, acompanhar e registrar pagamentos relacionados a orçamentos e agendamentos, por meio de um gateway de pagamento externo. A especificação descreve as chamadas entre o usuário, as interfaces do sistema, o backend, o banco de dados e o gateway, com atenção especial aos retornos assíncronos (webhooks) que informam o resultado das transações.

O objetivo é mapear e documentar o funcionamento da integração, não implementar nem contratar um serviço financeiro. O gateway permanece indefinido e os endpoints e exemplos apresentados são contratos propostos, sujeitos à adaptação quando o provedor for escolhido.
Observação de escopo: o MVP inicial previa pagamentos fora da plataforma. Portanto, o processamento online descrito neste documento representa uma funcionalidade futura ou uma alteração de escopo a ser aprovada, não uma implementação existente.

13.1. Contexto arquitetural e responsabilidades

A integração seguirá a arquitetura já planejada para o OrbiVet: backend em Java com Spring Boot, organizado como monólito modular e exposto por uma API REST; PostgreSQL como banco de dados; React com TypeScript na interface web e React Native com Expo na aplicação móvel.

Participante	Responsabilidade
Dono do animal	Selecionar o orçamento ou agendamento, iniciar o pagamento e consultar seu resultado.
Frontend OrbiVet	Acionar os endpoints do backend, abrir o checkout do gateway e exibir o status registrado.
Backend OrbiVet	Aplicar regras de negócio, impedir cobranças indevidas, solicitar o checkout, validar webhooks e atualizar os registros.
Banco de dados PostgreSQL	Armazenar pagamentos, vínculos com orçamentos/agendamentos, identificadores externos e histórico de eventos.
Gateway de pagamento	Disponibilizar o checkout, processar a transação e enviar notificações sobre mudanças de status.

Será adotado, para fins desta especificação, o modelo de checkout hospedado pelo gateway: o usuário informa os dados de pagamento em uma página segura do

provedor. O OrbiVet não recebe nem armazena dados completos de cartão. Essa escolha reduz a exposição de dados financeiros, mas não elimina as obrigações de segurança e privacidade aplicáveis.

13.2. Regras de Negócio

- O pagamento só poderá ser solicitado por um usuário autenticado e autorizado a pagar o registro correspondente.
- Um orçamento deverá estar aprovado, vigente e com valor definido. Alterações posteriores de valor exigirão nova autorização antes da cobrança.
- Um agendamento deverá existir e estar em uma situação que permita cobrança, conforme as regras do prestador. A confirmação financeira não substitui uma eventual aprovação do agendamento.
- O valor e a moeda serão obtidos e validados pelo backend com base no registro existente; o frontend não determina o valor cobrado.
- O sistema verificará se já existe uma cobrança em aberto ou um pagamento aprovado para a mesma obrigação financeira. Se orçamento e agendamento estiverem vinculados ao mesmo serviço, não será permitido cobrá-lo duas vezes.
- Uma tentativa repetida de criação não deverá gerar cobranças duplicadas. O backend reutilizará a cobrança válida quando apropriado e usará uma chave de idempotência nas operações externas, caso o gateway ofereça esse recurso.
- O redirecionamento para uma página de sucesso do checkout não comprova pagamento aprovado. A confirmação dependerá de consulta autenticada ao gateway, normalmente iniciada por um webhook.
- Webhooks poderão chegar com atraso, repetidos ou fora de ordem. O backend deverá manter o estado financeiro consistente e registrar eventos relevantes para auditoria.

13.3. Fluxo de funcionamento

Criação da cobrança

- O dono do animal seleciona Pagar em um orçamento aprovado ou em um agendamento elegível.
- O frontend envia POST /api/pagamentos ao backend, informando o tipo de origem e o identificador do registro.
- O backend autentica a solicitação, consulta o PostgreSQL e valida autorização, situação do serviço, valor, moeda e ausência de pagamento conflitante.

- Caso haja cobrança ativa reaproveitável, o sistema retorna seus dados. Caso exista pagamento já aprovado, impede uma nova cobrança para a mesma obrigação.
- Para uma cobrança nova, o backend registra o pagamento como PENDENTE e envia ao gateway uma solicitação de criação de checkout, com identificador interno de referência e chave de idempotência.
- O gateway retorna um identificador externo e uma URL segura de checkout. O backend registra esses dados e envia a URL ao frontend.
- O frontend abre o checkout, onde o usuário realiza a tentativa de pagamento.

Resultado do pagamento

- O gateway envia uma notificação POST ao endpoint de webhook configurado no OrbiVet quando ocorre uma mudança relevante.
- O backend valida a autenticidade da notificação e verifica se o evento já foi tratado.
- Para um evento novo e válido, o backend consulta o gateway pela API autenticada, confirmando a transação e conferindo identificador, valor, moeda e referência ao registro.
- Se os dados forem consistentes, o backend atualiza o pagamento e, quando couber, a situação financeira do orçamento ou agendamento vinculado.
- O frontend consulta GET /api/pagamentos/{id} e exibe o status existente no OrbiVet.

Importante: o usuário pode voltar ao OrbiVet antes de o webhook ser recebido. Nesse caso, o frontend exibirá PENDENTE e permitirá consultar novamente mais tarde, sem presumir aprovação.

Indisponibilidade e Conciliação

Se a criação da cobrança sofrer timeout ou perda de conexão, o OrbiVet não deverá repetir a operação de forma indiscriminada: a cobrança pode ter sido criada no gateway mesmo sem a resposta chegar ao backend. O caso será registrado como resultado externo ainda não confirmado, com posterior consulta ou repetição segura usando a mesma chave de idempotência.

Caso a notificação não seja entregue ou seu processamento falhe, deverá existir um mecanismo de conciliação para consultar transações pendentes e corrigir divergências com base no estado oficial retornado pelo gateway. O desenho detalhado e a frequência dessa rotina serão definidos na implementação.

13.4. Contratos propostos de API

As rotas abaixo pertencem ao OrbiVet, não ao gateway. A autenticação do usuário será aplicada às rotas de criação e consulta; o webhook terá autenticação própria por assinatura ou mecanismo equivalente do provedor.

Método	Rota/operação	Quem chama	Finalidade
POST	/api/pagamentos	Frontend → backend	Solicitar uma cobrança para um orçamento ou agendamento.
GET	/api/pagamentos/{id}	Frontend → backend	Consultar o status registrado para um pagamento autorizado ao usuário.
POST	/api/webhooks/pagamentos	Gateway → backend	Notificar evento relacionado à transação; sem sessão de usuário.
Operação externa	Criar checkout no gateway	Backend → gateway	Criar uma cobrança e obter seu identificador e URL.
Operação externa	Consultar pagamento no gateway	Backend → gateway	Verificar status e dados oficiais antes de atualizar o banco.

Exemplo de corpo enviado pelo frontend:

```
{
  "tipoOrigem": "ORCAMENTO",
  "origemId": 125
}
```

O valor AGENDAMENTO também poderá ser utilizado em tipoOrigem. É recomendável enviar um identificador de idempotência em um cabeçalho, por exemplo Idempotency-Key, cujo comportamento será definido pelo backend. A origem, a autorização e o valor serão validados no servidor.

Exemplo de resposta para cobrança criada (HTTP 201):

```
{
  "pagamentoId": 381,
  "status": "PENDENTE",
  "valor": 420.00,
  "moeda": "BRL",
  "checkoutUrl": "https://checkout.exemplo.invalid/sessao/abc123"
```

}

A URL utiliza um domínio reservado para exemplos e não é um endereço funcional. Uma cobrança já existente e válida poderá ser retornada com HTTP 200; uma solicitação inválida ou não autorizada receberá código 4xx apropriado. Em situação de resultado desconhecido no gateway, o sistema poderá retornar um estado de processamento sem fornecer checkout até a conciliação.

GET /api/pagamentos/{id}

Exemplo de resposta (HTTP 200):


```
{
  "pagamentoId": 381,
  "tipoOrigem": "ORCAMENTO",
  "origemId": 125,
  "valor": 420.00,
  "moeda": "BRL",
  "status": "APROVADO",
  "dataAtualizacao": "2026-10-29T19:00:00Z"
}
```


Somente o usuário vinculado ao pagamento ou um perfil expressamente autorizado poderá consultar seus dados. O endpoint não altera o status com base em informações fornecidas pelo navegador.

POST /api/webhooks/pagamentos

Exemplo conceitual de notificação recebida:


```
{
  "eventoId": "evt_001",
  "tipo": "pagamento.atualizado",
  "pagamentoGatewayId": "gw_841",
  "dataEvento": "2026-10-29T19:00:00Z"
}
```


Esse corpo é fictício: os campos, cabeçalhos de assinatura, tipos de eventos e formatos reais serão os da documentação do provedor selecionado. O OrbiVet não confiará isoladamente em um campo de status recebido na notificação.

Respostas previstas: HTTP 200 após processamento concluído ou recebimento durável para processamento posterior; HTTP 400/401 para notificação malformada ou assinatura inválida, conforme a integração; e HTTP 5xx para falhas temporárias internas que impeçam registrar/processar a notificação com segurança. A política de reenvio de eventos depende do provedor.

13.5. Estados e tratamento dos retornos

Estado interno	Significado	Comportamento do OrbiVet
PENDENTE	Cobrança criada ou aguardando confirmação financeira	Não marcar o serviço como pago; permitir consulta posterior.
APROVADO	Pagamento confirmado na API do gateway.	Registrar quitação financeira vinculada, sem aprovar automaticamente o atendimento.
RECUSADO	Tentativa negada.	Registrar a falha e permitir nova tentativa conforme as regras de cobrança.
CANCELADO	Cobrança cancelada pelo usuário, sistema ou provedor.	Encerrar a tentativa sem marcar o serviço como pago.
EXPIRADO	Prazo de pagamento encerrado.	Encerrar a cobrança e permitir nova criação quando aplicável.
ESTORNADO	Valor previamente aprovado foi revertido.	Registrar reversão e sinalizar ajuste financeiro e eventual análise do serviço.

Mapeamento: nomes e estados externos variam entre gateways. O backend traduzirá os resultados do provedor para o conjunto interno acima. ESTORNADO é documentado para consistência futura, mas o fluxo de solicitação de estornos não faz parte desta atividade.

Não se deve regredir automaticamente um pagamento de APROVADO para PENDENTE por causa de um evento antigo. Atualizações financeiras posteriores,

como estorno, deverão ser tratadas como eventos distintos e verificáveis. Eventos repetidos não devem produzir efeitos financeiros repetidos.

13.6. Persistência e rastreabilidade

Propõe-se acrescentar a entidade Pagamento, relacionada ao orçamento ou agendamento que originou a cobrança. Os atributos abaixo são sugestões para a modelagem, e não alteram os campos já existentes nessas entidades.

Campo proposto	Tipo sugerido	Finalidade
id	BIGINT ou UUID	Identificador interno do pagamento.
tipo_origem	VARCHAR ou enum	ORCAMENTO / AGENDAMENTO.
origem_id	BIGINT ou UUID	Identificador do registro de origem.
pagador_id	BIGINT ou UUID	Usuário responsável pela cobrança.
valor	NUMERIC(12,2)	Valor congelado para a tentativa.
moeda	CHAR(3)	Código da moeda, como BRL.
status	VARCHAR ou enum	Estado financeiro interno.
gateway	VARCHAR	Provedor da transação.
gateway_payment_id	VARCHAR, anulável	Referência externa, quando disponível.
chave_idempotencia	VARCHAR	Controle de repetições seguras.
criado_em, atualizado	TIMESTAMPTZ	Datas de registro e atualização.

Para auditoria e deduplicação, recomenda-se também uma tabela EventoPagamento, com identificador do evento do gateway, pagamento associado, tipo de evento, momento de recebimento e resultado do processamento.

Nota de modelagem: os campos tipo_origem + origem_id representam uma associação lógica. Caso o projeto exija integridade referencial direta no PostgreSQL, o esquema poderá utilizar duas chaves estrangeiras opcionais (orcamento_id, agendamento_id) com uma restrição que permita exatamente uma delas por pagamento. A prevenção de pagamento duplo entre um orçamento e seu agendamento vinculado deve ocorrer por uma regra de obrigação financeira comum, e não apenas pela igualdade de origem_id.

13.7. Segurança, confiabilidade e tratamento de falhas

- Autorização: verificar no backend se o usuário pode pagar ou consultar a cobrança; proteger os endpoints com as regras de autenticação e permissões do sistema.
- HTTPS e credenciais: usar conexões seguras; manter tokens e segredos do gateway no servidor, fora do código-fonte e do frontend.
- Checkout externo: não persistir número completo de cartão nem código de segurança do cartão (CVV) no OrbiVet.
- Autenticidade do webhook: validar assinatura, janela de tempo e demais requisitos exigidos pelo gateway escolhido, sem presumir que o formato de assinatura seja igual entre provedores.
- Confirmação independente: consultar a API do gateway antes de modificar o estado financeiro e conferir identificador, valor, moeda e referência da cobrança.
- Idempotência e concorrência: controlar repetições de requisições, eventos duplicados e atualizações simultâneas por meio de chaves únicas, transações de banco e verificações de estado.
- Falhas temporárias: registrar erros, responder apropriadamente e permitir novas tentativas ou conciliação sem criar uma segunda cobrança involuntária.
- Auditoria e privacidade: armazenar somente os dados necessários para rastreabilidade e proteger informações pessoais e financeiras conforme as regras aplicáveis.

Tempo de resposta dos webhooks: a versão básica do diagrama mostra o processamento antes de responder HTTP 200. Se consultas ao gateway ou atualizações puderem ser demoradas, a implementação deverá evoluir para persistir duravelmente a notificação, responder rapidamente com 2xx e processar o evento de forma assíncrona, sem perder registros. O 2xx só deverá ser enviado quando o evento já estiver tratado ou seguramente registrado para tratamento posterior.

13.8. Casos de teste e critérios de aceitação

Cenário	Resultado esperado
Orçamento aprovado e vigente	Cobrança gerada com valor correto e URL de checkout.
Agendamento elegível	Cobrança associada ao agendamento correto.
Orçamento vencido ou não aprovado	Solicitação recusada; nenhuma cobrança externa criada.
Usuário sem permissão	Acesso negado e ausência de cobrança.
Mesma solicitação repetida	Não criar cobranças duplicadas.
Orçamento e agendamento do mesmo serviço	Não cobrar duas vezes a mesma obrigação.
Pagamento confirmado pelo gateway	Estado APROVADO e atualização financeira do serviço.
Pagamento recusado ou cancelado	Registrar resultado sem marcar serviço como pago.
Webhook com assinatura inválida	Rejeitar notificação; não atualizar status.
Webhook duplicado ou antigo	Não repetir efeitos nem regredir status validado.
Usuário volta antes do webhook	Mostrar PENDENTE até a confirmação oficial.
Timeout na criação ou falha do webhook	Registrar ocorrência e possibilitar conciliação sem duplicidade.

A task será considerada documentalmente concluída quando o fluxo, o diagrama de sequência, as chamadas propostas da API, os retornos de sucesso/falha e as regras de tratamento estiverem descritos e coerentes. Esses testes são critérios futuros de implementação, não resultados já executados.

13.9. Decisões pendentes e limites desta especificação

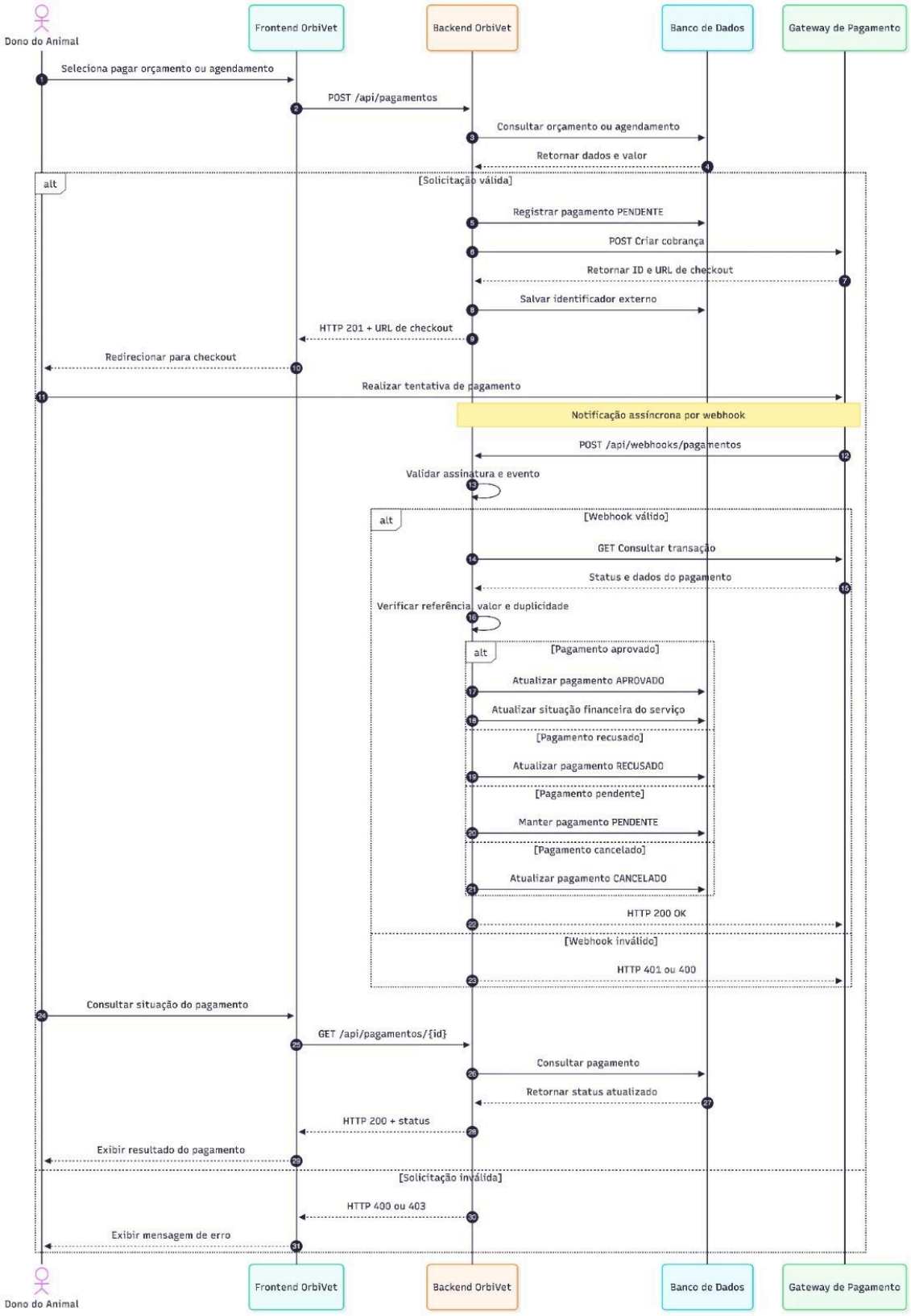
Antes de implementar, a equipe deverá definir: (a) o gateway; (b) quem receberá o dinheiro — plataforma, clínica ou prestador - e se haverá repasse; (c) quais métodos de pagamento serão aceitos; (d) cancelamentos, reembolsos e disputas; (e) a

obrigatoriedade de pagamento antecipado para agendamentos; e (f) quando a integração entrará no escopo do projeto.
Esta especificação não abrange processamento real, credenciais, conciliação contábil completa, divisão de recebíveis, assinaturas recorrentes ou implementação de reembolsos. Esses aspectos exigirão decisões de negócio e documentação complementar.

13.10. Referências Técnicas de Apoio

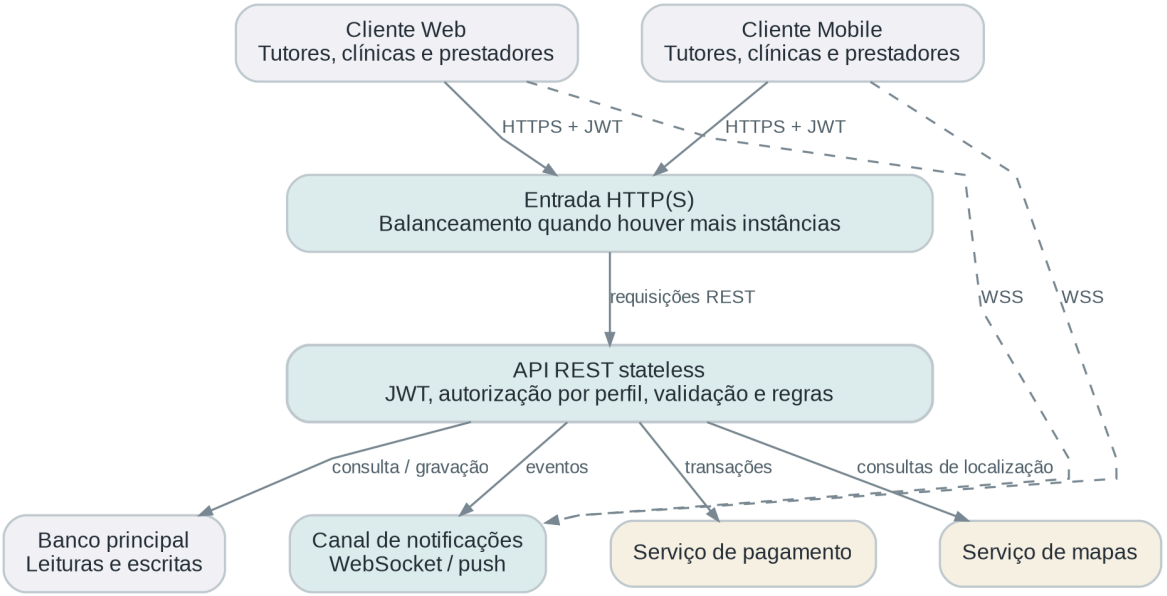
As referências abaixo apoiam as práticas gerais de validação e idempotência, sem indicar a escolha de um gateway específico:

Stripe — Recebimento e tratamento de webhooks.
Stripe — Requisições idempotentes.
Mercado Pago Developers — Webhooks.


Dono do Animal
Frontend OrbiVet
Backend OrbiVet
Banco de Dados
Gateway de Pagamento
1 Seleciona pagar orçamento ou agendamento
2 POST /api/pagamentos
3 Consultar orçamento ou agendamento
4 Retornar dados e valor
alt
[Solicitação válida]
5 Registrar pagamento PENDENTE
6 POST Criar cobrança
7 Retornar ID e URL de checkout
8 Salvar identificador externo
9 HTTP 201 + URL de checkout
10 Redirecionar para checkout
11 Realizar tentativa de pagamento
Notificação assíncrona por webhook
12 POST /api/webhooks/pagamentos
13 Validar assinatura e evento
alt
[Webhook válido]
14 GET Consultar transação
15 Status e dados do pagamento
16 Verificar referência, valor e duplicidade
alt
[Pagamento aprovado]
17 Atualizar pagamento APROVADO
18 Atualizar situação financeira do serviço
[Pagamento recusado]
19 Atualizar pagamento RECUSADO
[Pagamento pendente]
20 Manter pagamento PENDENTE
[Pagamento cancelado]
21 Atualizar pagamento CANCELADO
22 HTTP 200 OK
[Webhook inválido]
23 HTTP 401 ou 400
24 Consultar situação do pagamento
25 GET /api/pagamentos/{id}
26 Consultar pagamento
27 Retornar status atualizado
28 HTTP 200 + status
29 Exibir resultado do pagamento
[Solicitação inválida]
30 HTTP 400 ou 403
31 Exibir mensagem de erro
Dono do Animal
Frontend OrbiVet
Backend OrbiVet
Banco de Dados
Gateway de Pagamento

14.Consolidação dos Diagramas Arquiteturais

Definição Arquitetural do Sistema



Visão das interfaces, da API REST stateless, dos dados e das integrações.

14.1. Objetivo e critérios de consolidação

Este documento mostra como as partes do sistema vão se comunicar. Mostrando o caminho das informações entre o site, o aplicativo, o backend, o banco de dados e os serviços externos.

O site e o aplicativo vão acessar a API REST por HTTPS. O backend vai validar o usuário, conferir suas permissões, executar as regras do sistema e consultar ou salvar os dados. Os alertas vão utilizar também um canal de comunicação em tempo real.

Pontos considerados na arquitetura

- Backend REST stateless, validação de dados, autenticação, autorização e respostas em JSON.
- Login com JWT, permissões por perfil e funções de pets, saúde, alertas e agendamentos.
- Banco de dados principal, integrações de pagamento e mapas, além de WebSocket e notificações push.
- Crescimento gradual: mais instâncias de API, balanceamento de carga e réplicas de leitura conforme a necessidade.

Também pensando no crescimento do OrbiVet. No começo, o sistema vai funcionar com uma estrutura mais simples, para evitar gastos desnecessários. Mesmo assim, os componentes ficarão separados para facilitar melhorias e a expansão no futuro.

Os diagramas ajudam a acompanhar cada fluxo. No alerta de um pet perdido, por exemplo, a API confere os dados, registra a ocorrência e envia os avisos. No agendamento ou no histórico de saúde, o acesso depende das permissões do usuário. Assim, conseguimos mostrar o papel de cada parte do sistema.

Como ler os diagramas

Elemento	Significado
Caixa verde/azulada	Aplicação ou serviço interno do OrbiVet
Caixa cinza	Usuário, interface ou dado armazenado
Caixa bege	Integração externa ou recurso usado na expansão
Linha tracejada	Comunicação complementar ou prevista para o crescimento

14.2. Diagrama de contexto — Visão do Sistema

O primeiro diagrama mostra quem utiliza o OrbiVet e como a plataforma se conecta aos serviços externos. Tutores, clínicas, veterinários, petshops e prestadores vão acessar tanto o site quanto o aplicativo, com as funções e permissões próprias de cada perfil.

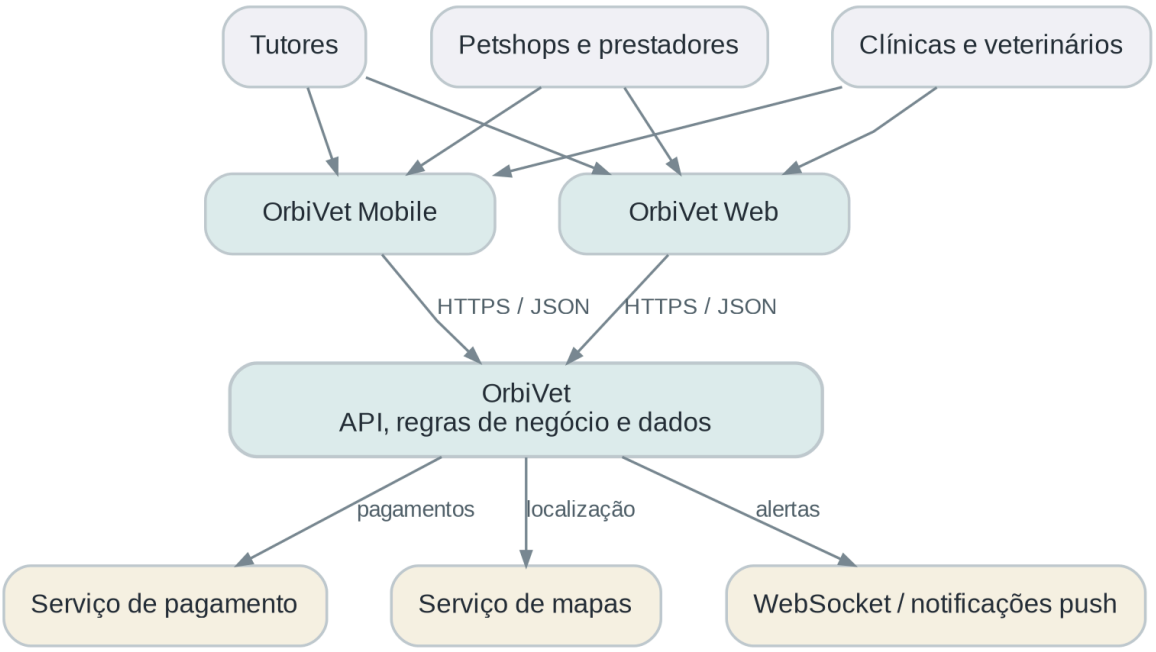


Figura 1 — Usuários, plataformas e integrações do OrbiVet.

O sistema vai centralizar os cadastros e as informações dos animais. A integração de mapas será usada na busca por estabelecimentos próximos, a integração de pagamento ficará responsável pelas transações financeiras e os canais de notificação entregarão avisos aos usuários.

Responsabilidades principais

- Tutores: cadastrar animais, acompanhar a saúde, publicar alertas e marcar serviços.
- Clínicas e veterinários: registrar atendimentos, exames, tratamentos e vacinações.
- Petshops e prestadores: gerenciar serviços e agendamentos disponíveis para seu perfil.
- OrbiVet: validar o acesso, aplicar regras de negócio e manter os registros do sistema.

14.3. Diagrama de contêineres — visão integrada

Este diagrama mostra como o site e o aplicativo vão se conectar ao backend. As duas interfaces vão utilizar a mesma API REST stateless, com autenticação por JWT e acesso controlado conforme o perfil do usuário.

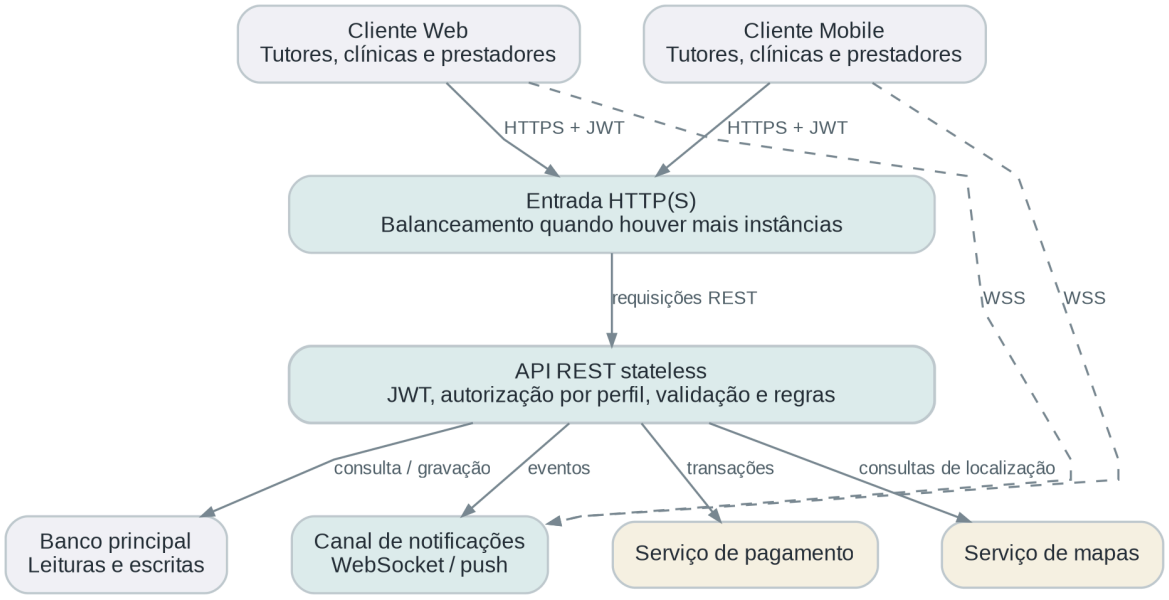


Figura 2 — Clientes, API, banco principal, notificações e serviços externos.

As requisições vão chegar pela entrada HTTPS e seguir para a API. O backend vai processar as operações e consultar ou atualizar o banco de dados. As notificações

vão seguir por WebSocket ou push, enquanto pagamentos e mapas ficarão ligados aos seus respectivos serviços.

O balanceamento de carga entra quando o sistema passar a ter mais de uma instância da API. Assim, não precisamos manter essa estrutura mais cara desde o começo.

14.4. Diagrama de componentes — backend REST stateless
O backend vai receber as requisições, identificar o usuário, conferir as permissões e executar as regras de negócio. Depois disso, realiza as consultas ou gravações necessárias. O diagrama separa essas responsabilidades para mostrar o funcionamento interno da API.

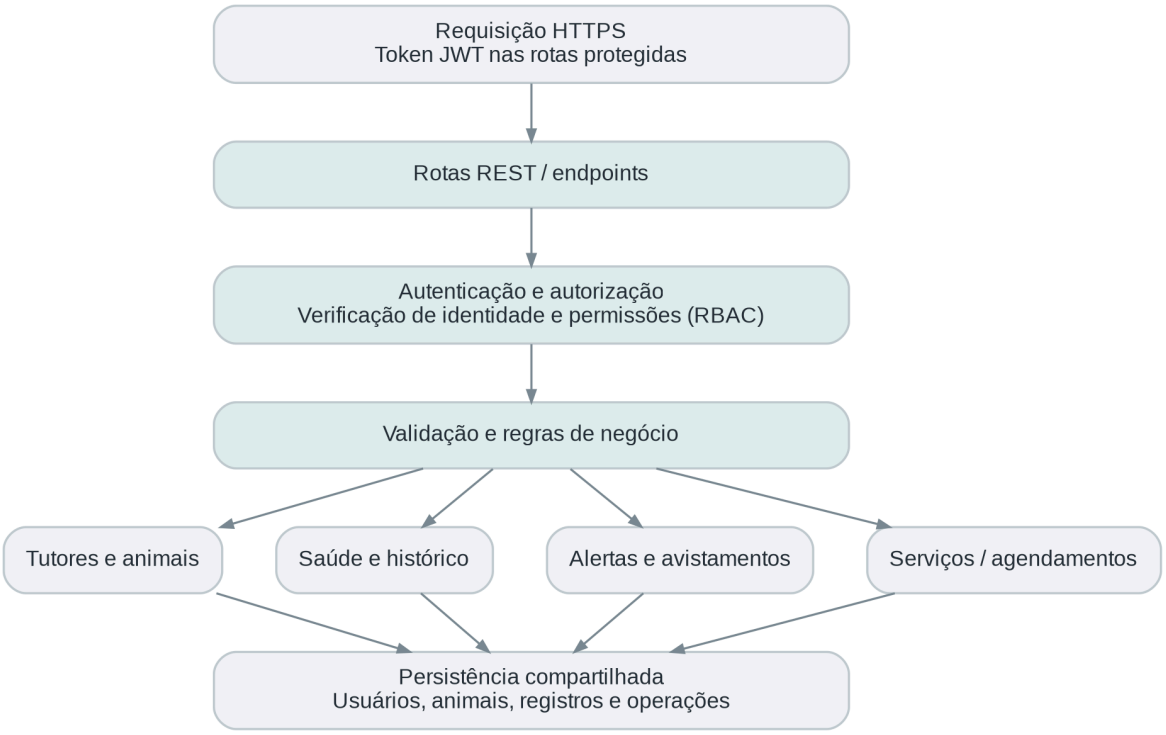


Figura 3 — Etapas de uma requisição e recursos atendidos pelo backend.

Principais rotas da API

Recurso / ação	Rota
Login	POST /api/auth/login
Cadastro de tutor e animal	POST /api/tutores | POST /api/animais
Histórico de saúde	GET /api/animais/{idAnimal}/historico
Alerta de pet perdido	POST /api/alertas
Estabelecimentos próximos	GET /api/estabelecimentos/proximos
Agendamento	POST /api/agendamentos
Registros de saúde	POST /api/exames | POST /api/prescricoes | POST /api/vacinacoes

15.5. Fluxo de alerta e comunicação em tempo real

O alerta de desaparecimento é um exemplo de como a API, o banco e as notificações vão trabalhar juntos. O tutor registra o caso pelo site ou aplicativo, o backend salva as informações e o aviso é enviado para outros usuários da região.

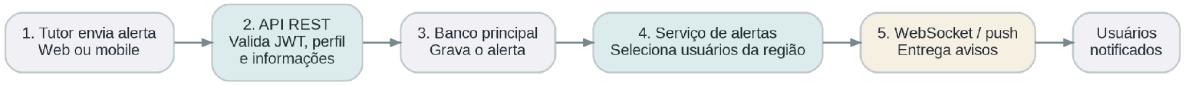


Figura 4 — Etapas do cadastro de um alerta e envio das notificações.

Etapas do fluxo

- O tutor envia POST /api/alertas com dados do pet, foto e localização do desaparecimento.
- A API confere o JWT, as permissões e os dados recebidos.
- O backend registra o alerta no banco principal.
- O sistema identifica os usuários que devem receber o aviso.
- A notificação é entregue por WebSocket aos clientes conectados e por push quando aplicável.

Relação com o stateless

O cadastro do alerta continua sendo uma requisição REST independente. O WebSocket mantém uma conexão própria para entregar avisos. Quando houver mais servidores, esse canal também terá de distribuir os eventos entre as instâncias.

14.6. Escalabilidade e infraestrutura lógica

O OrbiVet vai começar com uma instância da API e um banco principal. Essa escolha mantém os custos menores no início, mas não altera o funcionamento stateless do backend. Com mais usuários e acessos, a infraestrutura será ampliada de forma gradual.

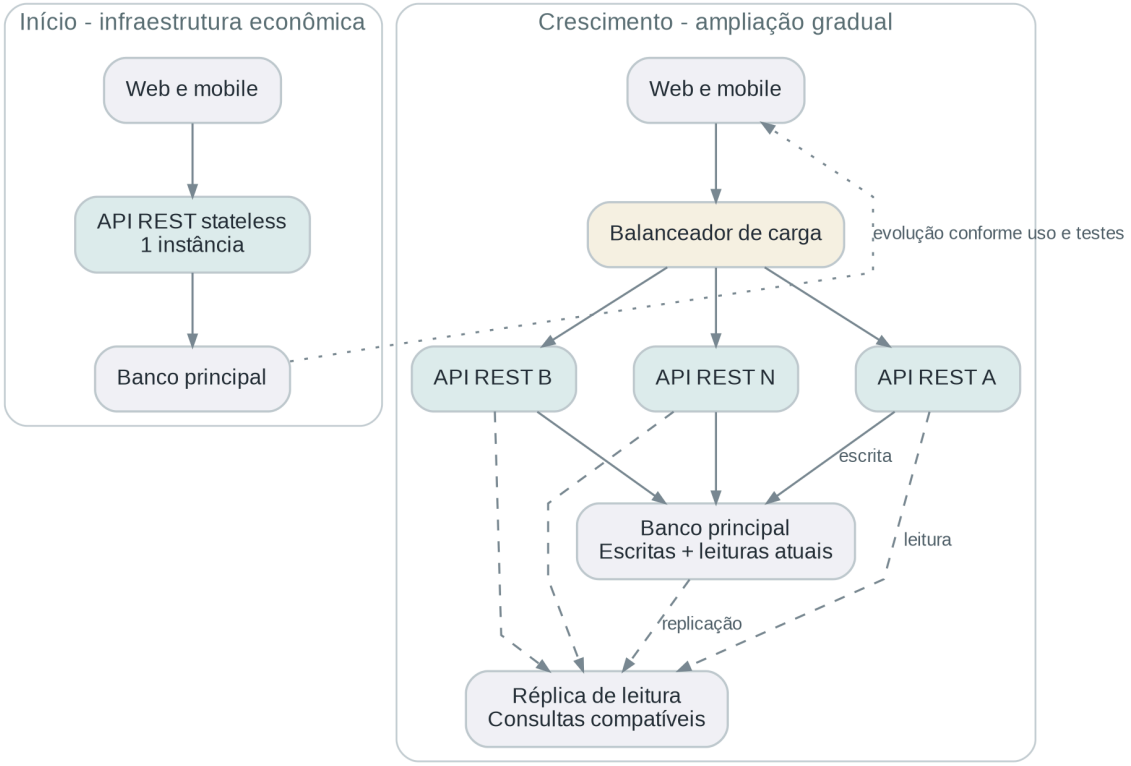


Figura 5 — Estrutura inicial e evolução para mais instâncias e réplicas.

Quando houver mais de uma instância, o balanceador vai distribuir as requisições entre elas. O banco principal continuará recebendo as escritas e as leituras que precisam de dados atualizados. As réplicas serão usadas para distribuir consultas que aceitem um pequeno atraso na atualização.

Implantação de baixo custo

A expansão vai acompanhar o uso real da plataforma e os resultados dos testes de carga. Assim, os recursos extras só entram quando forem necessários, sem contratar vários servidores desde a primeira versão.

14.7. Mapeamento de integrações

O OrbiVet vai integrar pagamentos, mapas e notificações às funções do sistema. Essas integrações ficam separadas da API principal para organizar melhor o fluxo das informações.

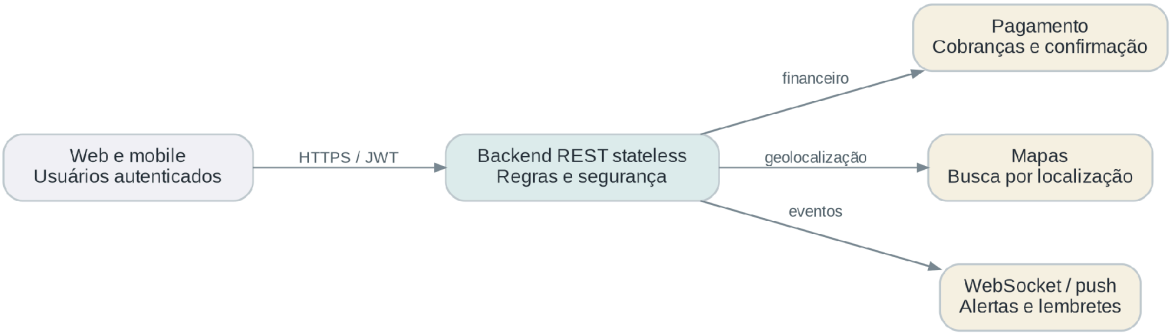


Figura 6 — Integrações utilizadas pelo backend do OrbiVet.

Integração	Função no OrbiVet
API de pagamento	Processar cobranças relacionadas aos serviços e retornar o resultado ao sistema.
Mapas e geolocalização	Mostrar clínicas e estabelecimentos próximos da localização informada.
WebSocket / push	Enviar alertas de animais perdidos, avisos e lembretes aos usuários.

Segurança e proteção de dados

A autenticação vai utilizar JWT no cabeçalho Authorization. O backend vai verificar as permissões de cada perfil antes de liberar o acesso a dados e operações. A comunicação vai usar HTTPS, e os usuários terão recursos para consultar, corrigir e excluir informações pessoais, seguindo os cuidados previstos pela LGPD.

14.8. Atendimento aos requisitos corporativos

Requisito	Figuras	Como será atendido
Web e mobile	1 e 2	As duas interfaces acessam a API por HTTPS, com acesso por perfil.
Backend REST stateless	2 e 3	Requisições processadas sem depender de sessão local.
Persistência	2, 3 e 5	Banco principal mantém os registros do sistema.
Escalabilidade horizontal	5	Novas instâncias e balanceador conforme a demanda.
Leituras e escritas	3 e 5	Escritas no primário e consultas distribuídas conforme a necessidade.
Alertas em tempo real	2 e 4	Comunicação por WebSocket e notificações push.
Pagamentos e mapas	1, 2 e 6	Integrações acessadas a partir do backend.
Segurança	2 e 3	JWT, controle de permissões e comunicação por HTTPS.

14.9. Conclusão

Os diagramas mostram como o OrbiVet vai funcionar e como as principais partes do sistema se conectam. O site e o aplicativo vão usar a mesma API REST, enquanto o backend vai cuidar das regras de negócio, das permissões e do acesso ao banco de dados.
Os alertas terão comunicação em tempo real e as integrações vão atender pagamentos e localização. A infraestrutura começará simples, com uma instância da API e um banco principal, e crescerá conforme o número de usuários e o volume de operações aumentar.